PRÁCTICA FINAL · GESTOR DE CITAS

Módulo: MF0491_3 · Programación web en el entorno cliente
Duración: 4 horas
Modalidad: Individual
Tecnologías: HTML, CSS y JavaScript

Tu misión: digitalizar una peluquería

La peluquería necesita una aplicación que permita registrar las citas de sus clientes, mostrar los servicios y calcular sus importes.

Ya tienes preparados el diseño HTML, los estilos CSS y las imágenes. Tu trabajo consiste en desarrollar JavaScript y completar algunas partes de HTML y CSS.

Archivos de trabajo:

- `index.html`: estructura de la página.
- `styles.css`: estilos de la aplicación.
- `app.js`: archivo JavaScript que debes programar.
- `images/`: imágenes de la aplicación.

Completa los retos en orden. Comprueba que cada uno funciona antes de continuar.

EMPEZAMOS

Mira lo que vamos a construir

https://actividadfinalcitas.vercel.app/

Lo primero será clonar el repositorio de github donde encontrarás los archivos iniciales de html, css y js.

En todos tendrás que hacer algún cambio.

RETO 1 · Mostrar las citas iniciales

Objetivo: crear las citas y mostrarlas en la página.

Paso 1. Crear el array

En `app.js`, crea un array llamado `citas` que contenga dos objetos con las propiedades `cliente`, `servicio` y `precio`.

Introduce estos datos:

Cliente	Servicio	Precio
Ana	Corte	25 €
Luis	Barba	15 €

Paso 2. Mostrar las citas

1. Selecciona el contenedor `#listaCitas` del HTML.
2. Crea una función llamada `mostrarCitas()`.
3. Recorre el array con `for...of`.
4. Muestra una tarjeta por cada cita con el nombre del cliente, el servicio y el precio.
5. Ejecuta la función al abrir la página.

Puedes utilizar `innerHTML` o `createElement()`.

Pistas: `querySelector()`, `for...of`, `innerHTML`, `textContent`, `append()`.

En el CSS tienes preparadas las clases `.cita` y `.precio`.

Comprueba: al abrir la página aparecen Ana y Luis con sus datos correspondientes.

RETO 2 · Registrar nuevas citas

Objetivo: permitir que el usuario añada nuevas citas desde el formulario.

Paso 1. Validar el formulario con HTML

Abre `index.html` y añade los atributos necesarios para conseguir que:

- El nombre del cliente sea obligatorio.
- Sea obligatorio seleccionar un servicio.
- El precio sea obligatorio y no pueda ser negativo.

Pistas: `required`, `min`.

Paso 2. Programar el formulario

En `app.js`:

1. Selecciona el formulario y los tres campos.
2. Crea una función llamada `agregarCita()`.
3. Asocia la función al evento `submit` del formulario y evita que la página se recargue.
4. Lee los valores de los campos y convierte el precio a número.
5. Crea un objeto `nuevaCita` con las propiedades `cliente`, `servicio` y `precio`.
6. Añade la nueva cita al array utilizando `push()`.
7. Actualiza el listado, limpia los campos y muestra un mensaje de confirmación.

Pistas: `.value`, `Number()`, `push()`, `addEventListener()o onclick`,

Paso 3. Actualizar el contador

En el HTML tienes un elemento con el identificador `#totalCitas`.

Modifica `mostrarCitas()` para que muestre cuántas citas hay utilizando la propiedad `length` del array.

Comprueba:

Añade la siguiente cita:

Cliente	Servicio	Precio
Marta	Peinado	30 €

Deben aparecer tres citas y el contador debe mostrar **3**.

Intenta también enviar el formulario con campos vacíos. El navegador debe impedirlo.

RETO 3 · Calcular el importe total

Objetivo: calcular la suma de los precios de todas las citas. Mete esta función dentro de mostrarCitas para que cada vez que añada una cita se actualice el contador

En el HTML ya tienes preparado un apartado con el identificador `#importeTotal`.

Instrucciones

1. Selecciona el elemento `#importeTotal`.
2. Crea una función llamada `calcularImporte()`.
3. Declara una variable acumuladora que comience en cero.

4. Recorre el array `citas` y suma los precios.
5. Muestra el resultado acompañado del símbolo €.
6. Haz que el importe se actualice cuando añadas una nueva cita.

Pistas: `for...of`, `+=`, `textContent`.

Comprueba:

- Ana (25 €) + Luis (15 €) = **40 €**.
- Al añadir Marta (30 €), el importe debe actualizarse a **70 €**.

RETO 4 · El rincón de tu peluquería

Objetivo: crear una sección nueva utilizando HTML y CSS y darle funcionamiento con JavaScript.

Paso 1. HTML

Debajo del gestor de citas, y antes de la imagen de despedida, añade una sección con el título:

El rincón de tu peluquería

Incluye dos botones:

- «Servicio de la semana».
- «Consejo de peluquería».

Añade también un párrafo vacío donde se mostrarán los mensajes.

Asigna identificadores a los elementos que necesites seleccionar desde JavaScript.

Paso 2. CSS

Diseña la sección para que combine con la aplicación.

Debe tener un fondo diferente, separación entre los elementos y botones con colores y efecto `:hover`.

Paso 3. JavaScript

1. Selecciona los dos botones y el párrafo.
2. Crea una función para mostrar el servicio de la semana y otra para el consejo.
3. Programa los eventos `click` de ambos botones.
4. Muestra los mensajes en el párrafo mediante `textContent`.

Botón «Servicio de la semana»:

«Esta semana: tratamiento hidratante especial por 20 €».

Botón «Consejo de peluquería»:

«Consejo: utiliza agua fría para el último enjuagado.».

Pistas: `querySelector()`, `addEventListener()`, `textContent`.

Comprueba: al pulsar cada botón aparece el mensaje correspondiente, sin recargar la página.

Si has llegado hasta aquí enhorabuena!!!

Entrega de la práctica

1. Comprueba que la aplicación funciona y revisa la consola para detectar posibles errores.
2. Guarda `index.html`, `styles.css`, `app.js` y la carpeta `images` dentro de una carpeta del proyecto.
3. Comprime la carpeta completa en un archivo `.zip`.
4. Entrega el ZIP en el espacio indicado.

Antes de terminar…

Comprueba que sabes explicar:

- Qué información contiene el array de citas.
- Qué ocurre cuando utilizas `push()`.
- Por qué debes actualizar el DOM después de añadir una cita.
- Cómo utilizas un bucle para calcular el importe total.
- Qué función ejecuta cada botón.

Recuerda: puedes consultar tus apuntes y utilizar el autocompletado. Lo importante es que entiendas el código que escribes y sepas comprobar su funcionamiento.

¡Enhorabuena por llegar hasta aquí y a seguir programando!